CareSphere Viva Walkthrough

Sep 28, 2026 · @Suheb

The one idea behind these changes

If the database already knows something, the website shows the database's answer instead of a number or name someone typed in. This is the thread that connects every change below, and it is the best one-line answer to "how do you stop fake data?"

Before, several parts of the site were typed straight into the HTML: the homepage counters (5,000 volunteers, 1,00,000 lives), the Impact Dashboard cards and charts, the homepage testimonials with stock photos, and the Hall of Fame names. None of these changed when the real data changed, so the site could say things that weren't true.

Now there are only two kinds of figure on the site:

- **Counted figures.** Things the system can count from its own records, such as active volunteers, events held, verified hours and verified donations. Nobody types these; the server counts them every time the page loads.
- **Recorded figures.** Things the system cannot know by itself, such as trees planted or people reached. An admin types these on the Impact Numbers page, and each one shows the date it was last updated, so a reader can see how fresh it is.

The same rule is applied to people: a Hall of Fame entry or an admin promotion has to point at a real volunteer record, and the server checks it, not just the web page.

Hall of Fame: tied to real volunteers

A Hall of Fame entry can only be made for a real, approved volunteer, and optionally for an event where their hours were verified. The admin picks both from dropdowns; nothing about the person is typed.

How it works on screen. On the admin Recognition page, choosing "Volunteer, Hall of Fame" shows a Volunteer dropdown listing approved volunteers with their district and verified hours. Picking one fills a second dropdown with only the events where that person has verified hours. The "What they did" line and the year fill in automatically, and the admin can reword them. Awards are different: they come from outside organisations, so they stay as typed text.

How it works in the database. The recognition table gained two columns: `user_id` (which volunteer) and `event_id` (which event). These are foreign keys, which means the database itself refuses a link to a volunteer or event that doesn't exist.

Why the server checks too. A web page can be bypassed by calling the API directly, so the rules live in the backend (`content.py`). Every time an entry is created or edited, the server checks:

1. The volunteer exists and has the volunteer role.
2. Their application is approved.
3. If an event is chosen, they have **verified** hours there. Registering is not enough, because registering doesn't prove they turned up.
4. The name shown is taken from their account. If someone sends a different name, the server ignores it.

Older entries typed by hand before this change show a "Not linked" tag in the admin table, so they can be fixed or removed.

Live figures: counted, not typed

The Impact Dashboard, the homepage counters and the login page all read the same live numbers from one new route, `GET /api/stats` , so the three pages can never disagree.

What the server counts (in `stats.py` , fresh on every request, nothing stored):

Figure	Counted as	Left out
Active volunteers	Accounts with the volunteer role, active, and an approved application	Pending or rejected applicants, deactivated accounts
Events held	Completed events, plus published events whose date has passed	Drafts and future events
Volunteer hours given	Total of verified hours	Hours still pending or rejected
Funds raised	Total of verified donations	Donations not yet verified

The charts on the Impact Dashboard are also built from records: verified donations per month for the last six months, a running total of approved volunteers by the month they were approved, and events by programme area. The old charts used invented numbers inside the JavaScript.

Typed figures (trees planted, beneficiaries reached, health camps, districts) still come from the admin's Impact Numbers page, and each card says "Updated" with its date. If an admin has typed a figure with the same name as a counted one, such as an old "Active volunteers" row, the counted one wins and the typed one is hidden, so the page never shows two answers to one question.

What changed on each page:

- **Impact Dashboard:** all seven hardcoded cards and the fake trend badges ("+8% this quarter") are gone. Each card now says either "Counted live from our records" or "Updated" with a date.
- **Homepage:** the counters, events, testimonials and news were all typed in. They now load from the database. Events show upcoming published ones, or "What We've Been Up To" with recent ones if nothing is coming up. Testimonials show approved stories only, with a letter avatar instead of a stock photo that pretended to be the writer. The Register button now actually registers a signed-in volunteer.
- **Login page:** the side panel numbers now come from the same live figures, and the test-accounts box that listed every password is removed.

If the backend is down, each section says it couldn't load. It never falls back to made-up numbers.

Volunteer to admin: earned, checked, agreed

A volunteer can apply to become an admin, but only if their record passes seven rules, two other admins vouch for them, and a Super Admin interviews them and approves. Admin access is a position of trust, so no single person, and no self-declared answer, can grant it.

The seven rules are counted by the server from the database (`promotion.py`). The volunteer cannot tick them; they only see the result, with their real numbers.

Rule	Default bar	Why it's there
Volunteering for long enough	2 years since approval	Knows how the NGO works
Verified hours	100 hours	Real contribution, confirmed by an organiser
Events worked	12 events with verified hours	Experience across many events, not one long one
Programme areas covered	All 3 (Education, Healthcare, Green Cover)	Understands every programme they would manage
Active month after month	Verified hours in 9 of the last 12 months	Steady commitment, not a burst long ago
Turns up when signed up	90% attendance	Reliable: registered and then actually came
Clean record	No logged hours ever rejected	Honest with their own records

A Super Admin can change these numbers from the "Eligibility rules" button on the Admin Requests page, without editing any code. They are stored in their own one-row table, `promotion_rules`.

The flow, step by step:

1. **Check.** The volunteer opens "Become an admin" and sees the seven rules with ticks or crosses, for example "Events worked: 4 of 12". The form stays locked until all seven pass.
2. **Apply.** They write why they would make a good admin, their achievements, roles they have held, hours a week they can give, and a preferred programme. The server checks the seven rules again and refuses if any fail. Only one request can be waiting at a time.
3. **Endorse.** Other admins read the request next to the volunteer's live record and click Endorse with a written reason. The database stops the same admin endorsing twice.
4. **Interview.** A Super Admin talks to the volunteer and records interview notes. Plain Admins cannot do this step.
5. **Decide.** A Super Admin approves or rejects. Approve stays locked until: the record still passes all seven rules, there are two endorsements from admins **other than the one approving**, and the interview notes exist. Rejecting requires a written reason, which

the volunteer sees.

6. **Result.** On approval the server changes the account's role from volunteer to admin. Their volunteer history stays in the database.

Why the approver's own endorsement doesn't count: otherwise one Super Admin could endorse and approve alone, and the two-person check would be meaningless.

Demo account: Kavya Rao (`kavya.r@example.com`) was added by `add_veteran.py` with 2 years 6 months of history, 13 events, 104 verified hours, all three areas and 100% attendance, so the full flow can be shown. Her history is built relative to today's date, so she keeps qualifying whenever the script is run.

Sign-in roles: the dropdown must match

The "Sign in as" dropdown on the login page is checked against the role stored in the database, and a mismatch is refused. The role always comes from the database; the dropdown only has to agree with it.

Account	Picks	Result
Super Admin	Super Admin	Signed in
Super Admin	Admin	Refused: "That's a Super Admin account"
Admin	Super Admin	Refused: "Not a Super Admin account"
Volunteer	Admin or Super Admin	Refused: "That's a volunteer account"

The last missing case (a Super Admin picking "Admin" and getting in) was fixed in `auth.js` . When a sign-in is refused, the login token the server just issued is thrown away, so nothing is left signed in.

Promotion while signed in. If a Super Admin promotes or demotes someone who is already signed in, every admin page now asks the server "who is this really?" when it loads. If the role has changed, the page reloads once so the menu, the role label and the Super-Admin-only buttons match the database. If the account has been deactivated, they are signed out.

Hiding a button is not the protection. The pages hide buttons a person cannot use, but every action is also checked on the server. Someone who edits their browser session to pretend to be a Super Admin still gets a "403 Forbidden" from the API.

Smaller changes

- **Gallery removed.** Photos already appear with events, programmes and news, and there was never a public gallery page, only an admin one. The admin page, its sidebar link, its backend routes and its database table were all removed. A migration dropped the `gallery` table.
- **Newsletter now saves.** The "Stay Updated" box used to say "You're subscribed" without saving anything. Emails now go into a new `subscribers` table. Each email is stored in lower case and must be unique, so signing up twice never makes a duplicate; the second time it says "You're already on the list". Admins see the list on a new Subscribers page and can copy all active emails for a BCC line. Only a Super Admin can delete an entry.
- **Partnerships page fixed.** The admin Partnerships page loaded a script that did not exist, so its table was always empty. The script now lists requests from the "Partner With Us" form, lets an admin assign a request, keep internal notes, and move it from New to Contacted to Active partner or Declined. "Draft in Gmail" opens a reply worded to match the status.
- **Testimonials.** Anyone can send a story from the public site, but nothing appears until an admin approves it. The public API only ever returns approved stories. A testimonial table column (`user_id`) was also added so a story can be linked to a volunteer account in future.

Likely examiner questions

Short answers you can give out loud. Each points back to a section above for the detail.

Question	Answer
How do you stop fake data on the site?	If the database knows it, the page shows the database's answer. Figures are counted live; names in the Hall of Fame and admin promotions must point at real volunteer records, checked on the server.
What if someone skips your web page and calls the API directly?	Every rule is checked in the backend too. The page only hides buttons for convenience; the server refuses anything not allowed, with a 403 or 422.

Question	Answer
Where does "Active volunteers" come from?	The server counts accounts with the volunteer role that are active and have an approved application, every time the page loads. Nothing is stored or typed.
Why not let admins type every number?	Typed numbers go stale and can be wrong. We only type what the system can't know, like trees planted, and show the date it was last updated.
Why verified hours and not registrations?	Registering only shows intent. Verified hours are confirmed by an organiser, so they prove the person actually worked the event.
Why do you need two endorsements for an admin?	So no single person can grant admin access. The approving Super Admin's own endorsement doesn't count towards the two.
Can a volunteer fake their eligibility?	No. They can't enter any of the seven figures; the server counts them from hours, events and registrations that admins verified.
What stops the same admin endorsing twice?	A unique constraint in the database on the pair (request, admin). The database itself refuses the second one.
Why keep the "Sign in as" dropdown if the server knows the role?	To show role-based access clearly. It isn't trusted: it must match the stored role or sign-in is refused.
What happens if a role changes while someone is signed in?	Each admin page re-checks the role with the server on load and reloads if it changed, or signs them out if deactivated.
How did you change the database without losing data?	With Alembic migrations: each change (new columns, new tables, dropping the gallery table) is a versioned script applied with `alembic upgrade head`.
What happens if the backend is down?	Each section says it couldn't load. The site never falls back to made-up numbers.